VIVA PREPARATION — COMPLETE MASTER GUIDE

DAA / Java Practicals + Machine Learning Practicals | Based on the uploaded programs and notebooks

Purpose: End-to-end preparation for tomorrow's viva. This guide contains theory, viva questions, code explanations, dry runs, comparisons, complexity, common mistakes, examiner cross-questions, and rapid-revision material.

Important: The explanations below are grounded in the uploaded practicals. Where two versions of a practical were uploaded, both are covered and the corrected/fixed version is explicitly identified.

How to use this tonight

1. Read the one-page 'What I must say' summary for each practical.
2. Understand the code flow, not just the final answer.
3. Practice the Viva Questions aloud. If you can answer them without looking, move on.
4. Do the dry-run examples on paper.
5. Before sleeping, revise the Rapid-Fire section and the comparison tables.

Master list

#	Practical	Main idea
1A	Fibonacci Recursive	Recursion, call tree, exponential time
1B	Fibonacci Iterative	Loop, constant extra space, linear time
2	Huffman Encoding	Greedy algorithm, min-heap, prefix codes
3	Fractional Knapsack	Greedy by profit/weight ratio
4	N-Queens	Backtracking, constraint checking
5	Quick Sort Analysis	Divide-and-conquer, deterministic vs randomized pivot
ML-1	Uber Fare Prediction	Regression, preprocessing, Haversine distance
ML-2	Email Spam Detection	KNN vs SVM classification
ML-3	Bank Churn Neural Network	MLP, scaling, imbalance, oversampling
ML-4	Diabetes KNN	KNN, imputation, scaling, choosing k
ML-5	Sales K-Means	Unsupervised clustering, elbow method, PCA

PART I — GENERAL THEORY QUESTIONS

DAA fundamentals

Q: What is an algorithm?
A: A finite, ordered sequence of unambiguous steps used to solve a problem.

Q: What makes an algorithm good?
A: Correctness, efficiency, termination, clear input/output, and reasonable use of time and memory.

Q: What is time complexity?
A: How the running time grows as input size n grows.

Q: What is space complexity?
A: How much additional memory an algorithm requires as n grows.

Q: What is Big-O?
A: An asymptotic upper-bound notation commonly used to describe worst-case growth.

Q: What is Big-Theta?
A: A tight asymptotic bound describing the growth rate from both upper and lower sides.

Q: What is Big-Omega?
A: An asymptotic lower bound.

Q: What is recursion?
A: A function calling itself on smaller/subproblems until a base case is reached.

Q: Why is a base case necessary?
A: It stops recursion; without it, recursion can continue indefinitely or cause stack overflow.

Q: What is divide and conquer?
A: Divide the problem into smaller subproblems, solve them recursively, and combine the results.

Q: What is greedy method?
A: At each step, choose the locally best option with the goal that these choices lead to a global optimum.

Q: What is backtracking?
A: Build a solution incrementally, and when a choice cannot lead to a valid solution, undo it and try another choice.

Q: What is dynamic programming?
A: Solve overlapping subproblems once, store their results, and reuse them.

Q: Greedy vs dynamic programming?
A: Greedy commits to local choices; DP considers subproblem relationships and stores results. Greedy is correct only when the problem has the required greedy-choice property.

Q: What is a priority queue?
A: An abstract data type where the highest/lowest priority element can be retrieved efficiently. In the Huffman practical it is a min-priority queue.

Q: What is a heap?
A: A tree-based data structure satisfying a heap-order property; a min-heap keeps the smallest key at the root.

Q: What is stable sorting?
A: A sort that preserves the relative order of equal-key elements.

Q: What is in-place sorting?
A: A sorting algorithm that uses O(1) or small auxiliary memory apart from recursion/implementation details.

Machine Learning fundamentals

Q: What is Machine Learning?
A: A field where models learn patterns from data to make predictions or decisions without explicitly coding every rule.

Q: Supervised vs unsupervised learning?
A: Supervised learning uses labelled target values; unsupervised learning works without a target label and discovers structure such as clusters.

Q: Classification vs regression?
A: Classification predicts discrete classes; regression predicts continuous numerical values.

Q: What is a feature?
A: An input variable used by the model.

Q: What is a target/label?
A: The output the model is trained to predict.

Q: Why split train and test data?
A: To train on one portion and evaluate generalization on unseen data.

Q: What is overfitting?
A: The model learns training-specific patterns/noise and performs poorly on unseen data.

Q: What is underfitting?
A: The model is too simple to capture important patterns in the data.

Q: Why scale features?
A: Algorithms based on distance or gradient optimization can be strongly affected by feature magnitude.

Q: What is standardization?
A: Transforming a feature approximately to mean 0 and standard deviation 1 using (x - mean)/standard deviation.

Q: What is a confusion matrix?
A: A table of actual vs predicted classes, giving TP, TN, FP and FN for binary classification.

Q: Accuracy?
A: (TP + TN) / total predictions.

Q: Precision?
A: TP / (TP + FP): among predicted positives, how many are truly positive.

Q: Recall?
A: TP / (TP + FN): among actual positives, how many were found.

Q: F1-score?
A: Harmonic mean of precision and recall: 2PR/(P+R).

Q: What is cross-validation?
A: Repeatedly split data into training/validation folds to estimate model performance more robustly.

Q: What is a hyperparameter?
A: A setting chosen before/during model training, such as K in KNN or regularization strength.

Q: What is data leakage?
A: Information from outside the training process, especially the test set, unintentionally influences model training or preprocessing.

Important formulas

Topic	Formula / key idea
Fibonacci	$F(n)=F(n-1)+F(n-2)$, $F(0)=0$, $F(1)=1$
Huffman	Repeatedly combine the two smallest frequencies
Fractional Knapsack	Value density = profit / weight
Haversine	Distance = $2R \cdot \text{asin}(\sqrt{\sin^2(\Delta lat/2)+\cos(lat1)\cos(lat2)\sin^2(\Delta lon/2)})$
Linear regression	$\hat{y} = b0 + b1x$
MAE	$\text{mean}(\lvert y - \hat{y} \rvert)$
MSE	$\text{mean}((y - \hat{y})^2)$
RMSE	sqrt(MSE)
Standardization	$z = (x - \mu)/\sigma$
KNN	Class is inferred from nearest training points
K-Means objective	Minimize within-cluster squared distance / inertia

PART II — DAA PRACTICAL 1: FIBONACCI

A. What this practical demonstrates

Two implementations of Fibonacci are uploaded: recursive and iterative. The recursive version counts function calls; the iterative version counts selected arithmetic/assignment operations. The recursive implementation uses the relation fibonacci(n-1)+fibonacci(n-2).

B. Code explanation — Recursive

```
static int stepCount = 0;

static int fibonacci(int n) {
    stepCount++;
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Code	Meaning
stepCount	Global/static counter used to count calls.
if n==0	First base case.
if n==1	Second base case.
fibonacci(n-1)+fibonacci(n-2)	Recursive branching into two smaller problems.

C. Dry run: n = 5

F(5)=F(4)+F(3); F(4)=F(3)+F(2); F(3)=F(2)+F(1). The same subproblems are recalculated many times. Result = 5.

The important viva point is not merely that the answer is 5; it is that recursion creates a tree of calls and repeats work.

D. Code explanation — Iterative

```
int a = 0, b = 1, c;
for (int i = 2; i <= n; i++) {
    c = a + b;
    a = b;
    b = c;
}
```

The loop keeps only the last two Fibonacci values. At every iteration, c becomes the next value, then a and b are shifted forward.

E. Recursive vs Iterative — must know

Feature	Recursive	Iterative
Technique	Recursion	Loop
Time	O(2^n) for naive recursion	O(n)
Extra space	O(n) call stack	O(1)
Repeated work	Yes	No
Code style	Short and mathematically natural	More efficient
Risk	Stack overflow for large n	No recursive stack growth

F. Theory questions

Q: What is Fibonacci series?
A: A sequence where each term is the sum of the previous two, starting here with 0 and 1.

Q: Why are two base cases used?
A: Because F(0) and F(1) are directly known and terminate the recursion.

Q: Why is recursive Fibonacci inefficient?
A: It repeatedly computes the same values, causing exponential growth in calls.

Q: Can recursive Fibonacci be improved?
A: Yes, memoization or dynamic programming reduces repeated computation to O(n).

Q: What is the space complexity of recursive Fibonacci?
A: O(n) due to the maximum recursion depth.

Q: What is the iterative space complexity?
A: O(1) auxiliary space.

Q: What happens for n=0 in the iterative program?
A: It directly prints 0 and sets the step count to 1.

Q: Why does iterative code start i at 2?
A: F(0) and F(1) are already represented by a=0 and b=1.

Q: Why use static stepCount?
A: The recursive method can update a shared counter across recursive calls.

Q: What is the recurrence relation?
A: T(n)=T(n-1)+T(n-2)+O(1).

G. Examiner cross-questions

Q: If I change n from 5 to 40, which version is safer?
A: Iterative, because it runs in linear time and uses constant extra space.

Q: Why not recursion if it is simpler?
A: For small n it is readable, but naive recursion becomes very expensive as n grows.

Q: Can Fibonacci be O(log n)?
A: Yes, with matrix exponentiation or fast doubling, though that is not the implementation here.

Q: What does stepCount mean exactly in your recursive code?
A: It counts function calls, including calls that immediately hit base cases.

PART III — DAA PRACTICAL 2: HUFFMAN ENCODING

A. Core idea

Huffman coding is a greedy compression technique. Characters with higher frequency receive shorter codes, while less frequent characters generally receive longer codes. The codes are prefix-free.

B. Code flow

6. Read the input text.
7. Count the frequency of every character using a HashMap.
8. Create a min-priority queue ordered by frequency.
9. Insert one node per character.
10. Remove the two smallest-frequency nodes.
11. Create a new internal node whose frequency is their sum.
12. Insert the new node back into the queue.
13. Repeat until one root remains.
14. Traverse left as 0 and right as 1 to generate codes.
15. Encode the original text by replacing every character with its Huffman code.

C. Important code

```
PriorityQueue<Node> pq = new PriorityQueue<>(
    Comparator.comparingInt(node -> node.frequency)
);
```

This creates a min-priority queue: the node with the smallest frequency is returned by poll().

```
Node left = pq.poll();
Node right = pq.poll();
Node newNode = new Node(
    left.frequency + right.frequency,
    left, right
);
pq.add(newNode);
```

This is the greedy step: always combine the two least frequent nodes.

D. Example

Suppose frequencies are A=5, B=2, C=1, D=1. First combine C and D → 2. Then combine B and CD → 4. Then combine 4 and A → 9. A possible code assignment depends on tie/order, but the important property is that no complete code is the prefix of another.

E. Theory + viva

Q: What is Huffman coding?
A: A lossless prefix-code compression algorithm that builds a binary tree using character frequencies.

Q: Why is it greedy?
A: At every step it chooses the two smallest-frequency nodes.

Q: Why use a min-heap?
A: It efficiently retrieves the two smallest frequencies.

Q: What is a leaf node?
A: A node with no left or right child; in this implementation it represents a character.

Q: What is a prefix code?
A: No valid codeword is a prefix of another codeword.

Q: Why is prefix-free important?
A: It makes encoded data uniquely decodable without separators.

Q: What does left=0 and right=1 mean?
A: The traversal convention used to assign binary codes.

Q: Is Huffman lossless?
A: Yes; the original symbols can be decoded exactly.

Q: What if two frequencies are equal?
A: Either tie ordering may be chosen; multiple valid Huffman trees/codes can exist with the same optimal cost.

Q: What data structure is central here?
A: A min-priority queue implemented by a heap.

Q: What is the time complexity?
A: Typically O(n log n) for n distinct symbols due to priority-queue operations.

Q: Why is frequency stored in internal nodes?
A: The sum of child frequencies determines the priority of the combined subtree.

Q: What does getOrDefault do?
A: Returns the existing count or a default value when the character has not appeared yet.

F. Common traps

- Do not say Huffman is a lossy compression method.
- Do not say the most frequent character always has a one-bit code; it depends on the tree.
- Do not confuse frequency with ASCII value.
- The exact bit codes can differ when frequencies tie, but compression optimality is preserved.

PART IV — DAA PRACTICAL 3: FRACTIONAL KNAPSACK

A. Core idea

The Fractional Knapsack problem allows an item to be divided. The uploaded program calculates profit/weight ratio, sorts in descending order, takes complete items while possible, and finally takes a fraction of the next item if necessary.

B. Key code

```
this.ratio = (double) profit / weight;

Arrays.sort(items,
    (a, b) -> Double.compare(b.ratio, a.ratio));
```

The explicit double cast prevents integer division. Sorting is descending because the highest value density should be considered first.

```
double fraction =
    (double) remainingCapacity / items[i].weight;
totalProfit += items[i].profit * fraction;
```

C. Dry run

Item	Weight	Profit	Profit/Weight
A	10	60	6
B	20	100	5
C	30	120	4

Capacity = 50. Take A fully (10), B fully (20), then 20/30 of C. Profit = 60+100+(20/30×120)=240.

D. Why greedy works here

Because the item can be divided. Taking the highest profit/weight ratio first is optimal for the fractional version.

E. Viva questions

Q: What is the objective?
A: Maximize total profit subject to knapsack capacity.

Q: Why use profit/weight ratio?
A: It measures profit gained per unit of capacity.

Q: Why can an item be split?
A: Because this is the fractional version.

Q: What if the item does not fit completely?
A: Take the fraction that fills the remaining capacity.

Q: Why is greedy optimal here?
A: Divisibility allows every unit of capacity to be assigned to the currently highest-value-density item.

Q: What is the time complexity?
A: O(n log n) because of sorting; the scan is O(n).

Q: What is the difference from 0/1 Knapsack?
A: 0/1 Knapsack either takes or rejects each item; fractional Knapsack can take part of an item.

Q: Can the greedy ratio method solve 0/1 Knapsack optimally?
A: No, not in general.

Q: Why use double?
A: The ratio and fractional profit can be non-integers.

Q: What happens when remainingCapacity becomes zero?
A: The loop breaks.

F. Fractional vs 0/1 Knapsack

Feature	Fractional	0/1
Split item?	Yes	No

Typical method	Greedy	Dynamic programming / branch and bound etc.
Ratio greedy optimal?	Yes	No, not generally
Result	May be fractional	Integral item selection
Typical DP complexity	Not needed for greedy solution	O(nW) for classic DP

PART V — DAA PRACTICAL 4: N-QUEENS

A. Core idea

N-Queens places N queens on an N×N chessboard so that no two queens attack each other. The uploaded program uses backtracking.

B. Safety checks

For a candidate (row, col), the program checks: same column, upper-left diagonal, and upper-right diagonal. Because queens are placed row by row from top to bottom, lower rows do not need checking yet.

C. Key code

```
if (isSafe(row, col)) {
    board[row][col] = 1;

    if (solveNQueens(row + 1))
        return true;

    board[row][col] = 0;   // Backtrack
}
```

If a placement eventually leads to a dead end, board[row][col] is reset to 0 and another column is tried.

D. Dry-run idea

For N=4, the program initially fixes a queen at row 0, column 0, then attempts row 1. If later rows cannot be completed, it backtracks and tries another safe column.

E. Viva questions

Q: What is backtracking?
A: A systematic trial-and-error technique that abandons a partial solution when it cannot lead to a valid complete solution.

Q: Why is isSafe required?
A: To ensure the new queen does not conflict with already placed queens.

Q: Why check only upper diagonals?
A: Earlier queens are in previous rows; future rows are empty at the moment of placement.

Q: What are the three attack directions checked?
A: Same column, upper-left diagonal, upper-right diagonal.

Q: What is the base case?
A: row == n, meaning all queens have been placed.

Q: What happens when a recursive call returns false?
A: The current queen is removed and another column is attempted.

Q: What is the worst-case complexity?
A: Commonly described as O(N!) for backtracking search, though actual work depends on pruning.

Q: What is the space complexity?
A: $O(N^2)$ for the board plus recursion depth O(N).

Q: Why is N=2 or N=3 unsolvable?
A: There is no arrangement satisfying all constraints.

Q: Why does your main fix the first queen at (0,0)?
A: That is an implementation choice in the uploaded program; it then searches for a solution consistent with that initial placement.

F. Important examiner trap

Do not say the program finds every possible N-Queens solution. The uploaded solveNQueens returns true on the first complete solution and stops. It finds one solution compatible with the fixed first queen, if one exists.

PART VI — DAA PRACTICAL 5: QUICK SORT ANALYSIS

A. Core idea

The uploaded practical implements two versions: deterministic Quick Sort always chooses the last element as pivot; randomized Quick Sort chooses a random index and swaps it into the last position before partitioning.

B. Partition logic

```
int pivot = arr[high];
int i = low - 1;

for (int j = low; j < high; j++) {
    deterministicComparisons++;
    if (arr[j] <= pivot) {
        i++;
        swap(arr, i, j);
    }
}
swap(arr, i + 1, high);
return i + 1;
```

The partition places values ≤ pivot on the left and larger values on the right, then places the pivot at its final position.

C. Randomized pivot

```
int randomIndex =
    low + rand.nextInt(high - low + 1);
swap(arr, randomIndex, high);
```

The random element is moved to the high position, so the same partition routine can be reused.

D. Deterministic vs randomized

Feature	Deterministic	Randomized
Pivot	Last element in this program	Random element
Worst-case time	$O(n^2)$	$O(n^2)$ theoretically
Expected time	Depends heavily on input/pivot rule	O(n log n) expected
Goal	Simple, predictable pivot rule	Reduce chance of repeatedly bad pivots
Extra randomness	No	Yes

E. Viva questions

Q: What is Quick Sort?
A: A divide-and-conquer sorting algorithm that partitions around a pivot and recursively sorts the two sides.

Q: What is a pivot?
A: The element used to partition the array.

Q: What does partition return?
A: The final index of the pivot after partitioning.

Q: Why can Quick Sort become $O(n^2)$?
A: If partitions repeatedly produce sizes n-1 and 0.

Q: When does that happen in this implementation?
A: For example, when the last element is repeatedly the smallest or largest element.

Q: Why randomize the pivot?
A: To reduce the likelihood of consistently poor partitions for adversarial or structured input.

Q: Does randomization remove the $O(n^2)$ worst case?
A: No, it changes the probability of encountering it.

Q: What is average/expected complexity?
A: O(n log n).

Q: What is the space complexity?
A: In-place partitioning uses O(1) auxiliary partition memory, while recursion requires stack space; average O(log n), worst O(n).

Q: Why clone the input array twice?
A: So both algorithms sort identical initial data and can be compared fairly.

Q: What does System.nanoTime measure here?
A: Elapsed wall-clock timing at high-resolution API granularity; it is not a guarantee of CPU-only time.

Q: Why might randomized execution time differ between runs?
A: The random pivot sequence can differ.

F. Code issue an examiner may notice

The program creates a new Random object inside randomizedPartition. That is functional for this practical, but in production-style code it is generally cleaner to reuse one Random instance rather than construct one for every partition call.

PART VII — ML PRACTICAL 1: UBER FARE PREDICTION

A. Objective

The notebook loads uber.csv, cleans the data, computes geographic trip distance using the Haversine formula, removes implausible observations, extracts time features, performs exploratory analysis, standardizes distance and fare, and trains Linear Regression to predict fare from distance.

B. Pipeline

16. Load CSV with pandas.
17. Inspect shape, info and null values.
18. Drop Unnamed: 0 and key; remove rows with missing values.
19. Compute Distance from pickup/dropoff latitude and longitude using Haversine.
20. Remove clearly invalid/outlier observations.
21. Extract Year, Month, Day, Day of Week, Hour.
22. Explore trips, fare, distance, correlations and heatmaps.
23. Set X = Distance and y = fare_amount.
24. Standardize X and y.
25. Split into 80% train and 20% test.
26. Fit LinearRegression.
27. Predict test values and calculate MAE, MSE, RMSE.
28. Inspect intercept/coefficient and regression plots.

C. Haversine explanation

```
km = 2 * 6371 * arcsin(
    sqrt(
        sin²(Δlat/2)
        + cos(lat1)cos(lat2)sin²(Δlon/2)
    )
)
```

Latitude and longitude are angular measurements, so the notebook first converts degrees to radians. 6371 is the approximate Earth radius in kilometers.

D. Why clean the data?

- Distance > 60 is removed.
- Distance == 0 is removed.
- Fare == 0 is removed.
- Fare < 0 is removed.
- Additional inconsistent distance/fare combinations are removed.

E. Linear regression

The model assumes a linear relationship between the input and target: fare ≈ intercept + coefficient × distance. The notebook uses only Distance as X for the regression model.

F. Metrics

Metric	Meaning	Viva interpretation
MAE	Average absolute error	Easy to interpret in target units
MSE	Average squared error	Penalizes large errors more strongly
RMSE	Square root of MSE	Same units as target; sensitive to large errors
R^2 / score()	Proportion of variance explained by the linear model	Higher is generally better on held-out data

G. Viva questions

Q: Why use Haversine instead of simple Euclidean distance?
A: Latitude/longitude are coordinates on the Earth's surface; Haversine computes great-circle distance more appropriately.

Q: Why convert degrees to radians?
A: The trigonometric functions use radians.

Q: What is the independent variable?
A: Distance in the regression model.

Q: What is the dependent variable?
A: fare_amount.

Q: Why standardize?
A: To put variables on a comparable scale; here both X and y are standardized before modeling.

Q: Why split the data?
A: To evaluate on unseen data.

Q: What is Linear Regression?
A: A supervised regression algorithm that models a linear relationship between input features and a continuous target.

Q: What does the coefficient represent?
A: The change in predicted standardized fare for a one-unit change in standardized distance, given this model.

Q: What is MAE?
A: Mean absolute difference between actual and predicted values.

Q: Why is RMSE often larger than MAE?
A: Because squaring errors gives more weight to large errors before taking the square root.

Q: What is an outlier?
A: An observation unusually far from the general pattern.

Q: Can fare be predicted perfectly from distance?
A: No; real fare depends on additional factors such as time, location, traffic, tolls and other effects.

PART VIII — ML PRACTICAL 2: EMAIL SPAM — KNN vs SVM

A. Objective

Classify email as Normal/Not Spam or Spam using K-Nearest Neighbors and Support Vector Machine, then compare performance.

B. Pipeline

29. Read emails.csv.
30. Inspect columns and missing values.
31. Drop rows with missing values.
32. Drop Email No. because it is an identifier.
33. Set X = features and y = Prediction.
34. Scale X using sklearn.scale.
35. Split into 70% training and 30% test data.
36. Train KNN with n_neighbors=7.
37. Predict and evaluate with accuracy and confusion matrix.
38. Train SVM with C=1.
39. Predict and evaluate with accuracy.

C. KNN

KNN is a lazy, instance-based classifier. It stores training examples and predicts a new sample from the labels of its nearest neighbors.

D. SVM

SVM attempts to find a decision boundary that separates classes with a large margin. C controls the penalty associated with misclassification in the standard soft-margin formulation.

E. KNN vs SVM

Feature	KNN	SVM
Learning style	Instance-based / lazy	Model-based / margin optimization
Main hyperparameter here	k=7	C=1
Distance sensitivity	Very high	Feature scaling often important
Prediction	Look at nearest points	Evaluate learned decision function
Training	Usually cheap compared with prediction	Can be more computationally intensive depending on data
Interpretability	Local neighbor reasoning	Margin/hyperplane reasoning

F. Viva questions

Q: Why scale data for KNN?
A: KNN uses distances, so large-scale features can dominate.

Q: Why scale for SVM?
A: Feature scale can affect the geometry of the margin and optimization.

Q: What does k mean in KNN?
A: Number of nearest neighbors considered for prediction.

Q: What happens if k is too small?
A: The model can be sensitive to noise and overfit.

Q: What happens if k is too large?
A: The model becomes too smooth and can underfit.

Q: What is C in SVM?
A: A regularization parameter controlling the trade-off between a wider margin and training misclassification.

Q: Why drop Email No.?
A: It is an identifier, not a meaningful predictive feature.

Q: Why use confusion matrix?
A: Spam detection has two classes and false positives/false negatives matter.

Q: What is a false positive here?
A: A normal email incorrectly classified as spam.

Q: What is a false negative?
A: A spam email incorrectly classified as normal.

Q: KNN training vs prediction cost?
A: Training is mostly storing data; prediction requires distance calculations to training points.

Q: Is KNN parametric?
A: It is generally considered non-parametric.

Q: Is SVM supervised?
A: Yes, it requires labelled examples.

PART IX — ML PRACTICAL 3: BANK CUSTOMER CHURN — NEURAL NETWORK

A. Two uploaded versions

Two versions were uploaded. The original notebook builds the neural network with Keras/TensorFlow and uses two hidden layers of 6 neurons plus a sigmoid output. A corrected FIXED notebook uses sklearn's MLPClassifier, adds explicit train/test stratification, training-only oversampling, L2 regularization and early stopping. The FIXED notebook is the more robust implementation and should be understood carefully.

B. Original Keras architecture

```
Input: 11 features
Hidden layer 1: 6 neurons, ReLU
Hidden layer 2: 6 neurons, ReLU
Output: 1 neuron, Sigmoid
Optimizer: Adam
Loss: binary_crossentropy
Epochs: 50
Batch size: 10
```

C. Fixed notebook pipeline

40. Load Churn_Modelling.csv.
41. Drop RowNumber, CustomerId and Surname because they are identifiers/non-predictive text.
42. Separate Exited as y.
43. One-hot encode Geography and Gender with drop_first=True.
44. Split 70/30 with stratify=y and random_state=42.
45. Fit StandardScaler on training data only and transform both sets.
46. Train baseline MLP with hidden layers (6,6).
47. Oversample the minority class on training data only.
48. Train improved MLP with hidden layers (64,32,16), alpha=0.001, Adam, early stopping.
49. Evaluate on the untouched test set using accuracy, confusion matrix and classification report.
50. Plot loss curve and compare baseline vs improved accuracy.

D. Neural-network concepts

Q: What is a neuron?
A: A computational unit that combines inputs with weights and bias, then applies an activation function.

Q: What is an activation function?
A: A nonlinear function applied to a neuron's weighted sum; it enables the network to learn nonlinear relationships.

Q: Why ReLU in hidden layers?
A: It is simple, computationally efficient, and helps neural networks model nonlinear relationships.

Q: Why sigmoid at the binary output?
A: It maps the output to a value between 0 and 1, suitable for binary probability-style prediction.

Q: Why binary cross-entropy?
A: It is a standard loss for binary classification.

Q: What is an epoch?
A: One complete pass through the training data.

Q: What is batch size?
A: Number of training examples processed before an optimizer update.

Q: What is Adam?
A: An adaptive gradient-based optimization algorithm.

Q: Why scale inputs?
A: Neural-network optimization generally benefits from features being on comparable scales.

Q: What is early stopping?
A: Stop training when validation performance stops improving, helping reduce overfitting.

E. Why improve the model?

The fixed notebook explicitly addresses class imbalance, model capacity, regularization and training duration. The test set is not oversampled so the final evaluation represents the original test distribution.

F. Baseline vs improved

Aspect	Baseline	Improved
Hidden layers	(6,6)	(64,32,16)
Alpha	0.0001	0.001
Max iterations	400	500
Early stopping	Yes	Yes
Class balance	Original training distribution	Minority oversampled in training only
Validation	15% internal validation	15% internal validation
n_iter_no_change	15	20

G. Viva questions

Q: What is churn?
A: A customer leaving the service.

Q: What is the target?
A: Exited: 1 for exited and 0 for stayed.

Q: Why drop Surname?
A: It is identifier/text information not intended as a predictive numeric feature.

Q: Why use one-hot encoding?
A: Neural-network input requires numeric features; categorical values must be represented numerically.

Q: Why drop_first=True?
A: It removes one redundant dummy category and avoids perfect linear dependence among dummy variables.

Q: Why stratify the split?
A: To preserve the class proportion in train and test.

Q: Why oversample only the training set?
A: Oversampling the test set would distort evaluation and leak information about the evaluation distribution.

Q: What is class imbalance?
A: One class occurs much less frequently than the other.

Q: What is L2 regularization?
A: A penalty on large model weights, controlled here by alpha, used to reduce overfitting.

Q: What does the confusion matrix show?
A: Actual vs predicted stayed/exited classes.

Q: What does predict() return in MLPClassifier?
A: Class labels.

Q: What does predict_proba() return?
A: Class probabilities; the fixed notebook's classification itself uses predict().

Q: Why not report a guessed accuracy?
A: The notebook states that actual accuracy and confusion matrix should be taken from the executed dataset rather than estimated in advance.

PART X — ML PRACTICAL 4: DIABETES — KNN

A. Objective

Predict the diabetes Outcome using KNN. The notebook performs data inspection, replaces medically implausible zero placeholders in selected columns with NaN, imputes missing values, scales features, tests k from 1 to 14, selects k=11 in the final classifier, and evaluates with a confusion matrix.

B. Data cleaning

The notebook treats 0 as missing for Glucose, BloodPressure, SkinThickness, Insulin and BMI, then imputes Glucose and BloodPressure with mean and SkinThickness, Insulin and BMI with median.

Why mean vs median?

Mean is sensitive to outliers; median is more robust. The notebook's chosen imputations should be explained as the implementation choice rather than presented as a universal rule.

C. K selection

```
for i in range(1,15):
    knn = KNeighborsClassifier(i)
    knn.fit(X_train,y_train)
    train_scores.append(knn.score(X_train,y_train))
    test_scores.append(knn.score(X_test,y_test))
```

The notebook evaluates k=1 through k=14, compares training and testing scores, and finally constructs KNN with k=11.

D. Viva questions

Q: Why replace zero values?
A: For selected medical measurements, zero is treated as a missing/invalid placeholder in this dataset context.

Q: Why impute instead of simply dropping rows?
A: Dropping many rows can unnecessarily reduce the dataset; imputation preserves observations.

Q: Why scale before KNN?
A: KNN relies on distance, so unscaled features can dominate.

Q: What is k?
A: Number of nearest neighbors used for classification.

Q: Why test many k values?
A: To compare generalization performance and choose a suitable neighborhood size.

Q: Why can k=1 overfit?
A: It can follow individual noisy training observations very closely.

Q: Why can a large k underfit?
A: It averages over too broad a neighborhood and may ignore local structure.

Q: What does stratify=y do?
A: Preserves the class distribution in train and test.

Q: What is the purpose of the confusion matrix?
A: Shows the counts of correct and incorrect predictions for each class.

Q: What is a decision region?
A: A region of feature space assigned to a particular predicted class.

E. KNN algorithm

51. Choose k.
52. Calculate distance from the query point to training points.
53. Select the k nearest points.
54. Count their class labels.
55. Assign the majority class.

The exact distance metric defaults to Euclidean distance in KNeighborsClassifier unless changed.

F. Important distinction

KNN is called a lazy learner because it does not build a strong parametric model during training; much of the work happens during prediction.

PART XI — ML PRACTICAL 5: K-MEANS + ELBOW + PCA

A. Objective

The notebook loads sales_data_sample.csv, removes unnecessary categorical/identifier columns, encodes selected categorical variables, converts PRODUCTCODE to numeric category codes, removes ORDERDATE, determines k using the elbow method, fits K-Means with k=3, and uses PCA to visualize the high-dimensional clusters in two dimensions.

B. Preprocessing flow

- Drop address/contact/status/location/identifier columns listed in the notebook.
- Inspect remaining categorical values in COUNTRY, PRODUCTLINE and DEALSIZE.
- One-hot encode PRODUCTLINE and DEALSIZE.
- Drop COUNTRY, PRODUCTLINE and DEALSIZE after encoding.
- Convert PRODUCTCODE to categorical integer codes.
- Drop ORDERDATE because month information is already represented in the data according to the notebook comment.
- Use the resulting numeric matrix for K-Means.

C. Elbow method

```
distortions = []
K = range(1,10)
for k in K:
    kmeanModel = KMeans(n_clusters=k)
    kmeanModel.fit(df)
    distortions.append(kmeanModel.inertia_)
```

Inertia/distortion measures within-cluster squared distances to centroids. It decreases as k increases. The notebook observes an elbow around k=3 and therefore fits KMeans(n_clusters=3).

D. K-Means

56. Choose k initial centroids.
57. Assign each point to the nearest centroid.
58. Recalculate each centroid as the mean of points assigned to it.
59. Repeat assignment/update until convergence or the stopping criterion is reached.

E. PCA

PCA is used after clustering to reduce the feature space to two principal components for visualization. It is not the clustering algorithm. K-Means is still fitted on X_train=df.values in the notebook; PCA is used for visualization of points and centroids.

F. Viva questions

Q: Is K-Means supervised?
A: No. It is an unsupervised clustering algorithm.

Q: What is a centroid?
A: The mean position of the points assigned to a cluster.

Q: What does k represent?
A: The number of clusters.

Q: What is inertia?
A: The sum of squared distances from each sample to its assigned cluster center.

Q: Why does inertia always tend to decrease as k increases?
A: More clusters give each point a closer or equal best-fitting centroid.

Q: Why use elbow method?
A: To identify a point after which additional clusters provide diminishing improvement.

Q: Why k=3 here?
A: The notebook's elbow plot shows an elbow around 3 and then a more gradual decrease.

Q: What is PCA?
A: Principal Component Analysis, a dimensionality-reduction method that creates orthogonal directions capturing variance.

Q: Does PCA create clusters?
A: No; in this notebook it is used to visualize the clusters.

Q: Why encode categorical data?
A: K-Means requires numeric vectors and distance calculations.

Q: Why one-hot encode PRODUCTLINE and DEALSIZE?
A: To represent categories numerically without imposing an artificial ordinal ranking.

Q: What is a drawback of K-Means?
A: It requires k in advance or an estimate, is sensitive to scaling/outliers, and assumes clusters are reasonably represented by centroid-based geometry.

Q: Why might scaling matter in K-Means?
A: Distance calculations can be dominated by high-magnitude features.

G. K-Means vs KNN

Feature	K-Means	KNN
Learning	Unsupervised	Supervised
Target label	Not required	Required
Purpose	Clustering	Classification/regression
k meaning	Number of clusters	Number of neighbors
Core operation	Centroid assignment	Nearest training examples
Training	Iterative centroid updates	Mostly stores labelled data

PART XII — ALL IMPORTANT DIFFERENCES

1. Recursive vs Iterative Fibonacci

Point	Recursive	Iterative
Control	Function calls	Loop
Repeated calculations	Many	None for basic sequence generation
Time	O(2^n) naive	O(n)
Space	O(n) stack	O(1)
Best viva phrase	Simple but expensive	Efficient and memory-friendly

2. Greedy vs Backtracking

Point	Greedy	Backtracking
Decision	Commit to local best	Try a choice and undo if needed
Typical example here	Fractional Knapsack / Huffman	N-Queens
Undo choices	No	Yes
Guarantee	Requires greedy-choice property	Searches solution space with pruning

3. Deterministic vs Randomized Quick Sort

Point	Deterministic	Randomized
Pivot	Fixed last element in your code	Random element
Expected	Input/pivot dependent	O(n log n) expected
Worst	$O(n^2)$	$O(n^2)$
Purpose	Simple comparison baseline	Reduce predictable bad partitions

4. KNN vs SVM

Point	KNN	SVM
Type	Supervised	Supervised
Idea	Neighbors	Maximum-margin decision boundary
Important parameter here	k=7 in spam notebook; k=11 in diabetes final model	C=1
Scaling	Important	Usually important
Prediction	Distance + voting	Evaluate learned model

5. Classification vs Regression

Point	Classification	Regression
Output	Class	Continuous number
Examples here	Spam, churn, diabetes	Fare
Metrics	Accuracy, precision, recall, F1	MAE, MSE, RMSE, R^2

6. KNN vs K-Means

Point	KNN	K-Means
Supervision	Supervised	Unsupervised
k	Neighbors	Clusters
Target	Known	Absent
Goal	Predict label	Discover groups

7. K-Means vs Hierarchical Clustering

The assignment statement mentions K-Means/hierarchical clustering, but the uploaded notebook implements K-Means and does not provide a hierarchical-clustering implementation. Therefore, do not claim that hierarchical clustering was implemented in the notebook unless your examiner asks the theoretical difference.

Point	K-Means	Hierarchical
Core idea	Centroids	Nested cluster tree/dendrogram
Need k	Usually yes	Can inspect dendrogram/cut later
Output	Partition	Hierarchy
Typical scalability	Good for large datasets	Can become expensive for large

		datasets

PART XIII — HOW TO EXPLAIN CODE TO THE EXAMINER

The 7-step answer pattern

60. State the objective.
61. State the input and output.
62. Explain preprocessing/data structures.
63. Explain the main algorithm.
64. Explain the important loop/decision.
65. State complexity and why the algorithm is suitable.
66. State what the output tells us.

Example: 'Explain this line'

```
Arrays.sort(items,
    (a,b) -> Double.compare(b.ratio, a.ratio));
```

Strong answer: 'This sorts the Item array in descending order of profit-to-weight ratio. The comparator compares b with a so the larger ratio comes first. This ordering is essential because Fractional Knapsack uses the greedy choice of highest value density first.'

Example: 'Explain this line'

```
X_train, X_test, y_train, y_test =
    train_test_split(X, y, test_size=0.3, random_state=42)
```

Strong answer: 'It divides the feature matrix and target into training and testing sets. 30% is reserved for testing. random_state=42 makes the split reproducible.'

Example: 'Explain this line'

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Strong answer: 'The scaler learns mean and standard deviation from training data only, transforms training data, and then applies the same learned transformation to test data. This avoids test-set leakage.'

PART XIV — EXAMINER'S TRICK QUESTIONS

Q: Why did you use this algorithm?
A: Answer using the problem type and algorithm property. Example: Fractional Knapsack permits splitting, so greedy by profit/weight is optimal.

Q: What happens if the input is very large?
A: Discuss time and memory complexity and whether the implementation scales.

Q: What if there are missing values?
A: Explain the actual handling in your notebook rather than giving a generic answer.

Q: What if the classes are imbalanced?
A: Accuracy alone may be misleading; consider precision, recall, F1 and class-balancing methods.

Q: Why random_state=42?
A: It makes randomized splitting/model initialization reproducible. The number 42 itself has no mathematical significance here.

Q: Why StandardScaler?
A: It standardizes features to mean approximately 0 and standard deviation approximately 1.

Q: Why not MinMaxScaler?
A: Either can be useful; the notebook uses StandardScaler. Standardization is especially common for algorithms sensitive to feature scale.

Q: What is data leakage?
A: Using information from test/future data during training or preprocessing.

Q: Why fit the scaler only on training data?
A: To prevent test-set statistics from influencing the model.

Q: Why is accuracy not enough?
A: It can hide poor performance on a minority class and does not distinguish types of errors.

Q: What is a false positive?
A: Predicted positive, actually negative.

Q: What is a false negative?
A: Predicted negative, actually positive.

Q: What is a hyperparameter?
A: A setting chosen by the practitioner/model-selection process rather than learned directly as a model weight.

Q: What is a parameter?
A: A value learned from training data, such as a linear regression coefficient or neural-network weight.

Q: What is normalization vs standardization?
A: Normalization is often used for rescaling to a bounded range; standardization specifically centers and scales by standard deviation. Terminology varies, so explain the exact transformation used.

Q: Why use PCA?
A: To reduce dimensionality, often for visualization or compact representation; in your K-Means notebook it is used for visualization.

If the examiner asks 'Why did you not use X?'

Use this structure: 'I chose Y because the problem has property Z. X is also possible, but it has trade-offs A/B. For this practical, Y is simpler/more suitable and matches the assignment requirement.' Never claim an algorithm is universally best.

PART XV — RAPID-FIRE LAST-MINUTE REVISION

Question	One-line answer
Recursive Fibonacci complexity?	O(2^n) time, O(n) stack for naive recursion.
Iterative Fibonacci complexity?	O(n) time, O(1) auxiliary space.
Huffman type?	Greedy, lossless, prefix coding.
Huffman main DS?	Min-priority queue / min-heap.
Fractional Knapsack key ratio?	Profit/weight.
Fractional Knapsack complexity?	O(n log n).
N-Queens technique?	Backtracking.
N-Queens safe checks?	Column + upper-left diagonal + upper-right diagonal.
Quick Sort average/expected?	O(n log n).
Quick Sort worst?	$O(n^2)$.
Randomized Quick Sort purpose?	Reduce likelihood of repeatedly poor pivots.
Uber model?	Linear Regression.
Uber target?	fare_amount.
Uber distance?	Haversine distance.
Uber regression metrics?	MAE, MSE, RMSE and model score/R^2.
Spam algorithms?	KNN and SVM.
Spam K?	7.
Spam SVM C?	1.
Diabetes algorithm?	KNN.
Diabetes final k?	11 in the uploaded notebook.
Why scale KNN?	Distance-based algorithm.
Churn target?	Exited.
Churn original network?	Keras Sequential, 6-6 hidden layers, sigmoid output.
Churn fixed network?	sklearn MLPClassifier; baseline (6,6), improved (64,32,16).
Churn improvement?	Minority oversampling + more layers/neurons + L2 + early stopping.
K-Means supervised?	No, unsupervised.
K-Means k?	3 in the uploaded notebook.
Elbow measures?	Inertia/distortion.
PCA purpose here?	2D visualization.
KNN vs K-Means?	KNN supervised classification; K-Means unsupervised clustering.

10 things you absolutely must be able to explain

67. Recursion and base case.
68. Greedy choice and why Fractional Knapsack allows it.
69. Huffman min-heap construction.
70. Backtracking and why a queen is removed after failure.
71. Quick Sort partition and pivot.
72. Why KNN needs scaling.
73. Why SVM uses C.
74. Why neural networks use activation functions.
75. Why K-Means needs an estimate of k and what the elbow means.
76. Why train/test preprocessing must avoid leakage.

PART XVI — MOCK VIVA: ROUND 1

Do this without looking at the answers. Give yourself 20–30 seconds per question.

Q: Explain your Fibonacci recursive program in 30 seconds.
A: It calculates the nth Fibonacci number using recursion. It has base cases for 0 and 1, and otherwise returns fibonacci(n-1)+fibonacci(n-2). A static counter counts every function call. Its main weakness is repeated computation, giving exponential time.

Q: Why is your iterative Fibonacci better for large n?
A: It avoids repeated recursive calls and keeps only the last two values, giving O(n) time and O(1) auxiliary space.

Q: Explain Huffman in 30 seconds.
A: I count character frequencies, put nodes in a min-priority queue, repeatedly combine the two least frequent nodes, build a binary tree, assign 0/1 by traversal, and encode the input using those prefix-free codes.

Q: Explain Fractional Knapsack.
A: I calculate profit/weight for each item, sort descending by that ratio, take full items while possible, and take a fraction of the next item when capacity is insufficient.

Q: Explain N-Queens.
A: I place queens row by row. Before placing, I check the column and two upper diagonals. If a later placement fails, I undo the current placement and try another column. That is backtracking.

Q: Explain Quick Sort partition.
A: I select a pivot, move elements <= pivot toward the left, place the pivot at i+1, and recursively sort the left and right partitions.

Q: Explain Uber practical.
A: I clean the Uber data, calculate trip distance using Haversine, remove invalid/outlier observations, extract time features, standardize distance and fare, split the data, fit linear regression, and evaluate with MAE, MSE and RMSE.

Q: Why KNN scaling?
A: Because KNN uses distance, so a feature with a larger numeric scale can dominate the neighbor calculation.

Q: Explain the churn neural network.
A: It predicts Exited. The data is cleaned and categorical features are encoded, features are standardized, the data is split, and an MLP is trained. The fixed version improves the baseline using training-only minority oversampling, more hidden layers, L2 regularization and early stopping.

Q: Explain K-Means and elbow.
A: K-Means assigns points to centroids and repeatedly updates those centroids. I calculate inertia for different k values; the elbow is where adding more clusters gives diminishing improvement. The notebook chooses k=3.

Mock Viva Round 2 — cross questions

Q: Why is Huffman called greedy but N-Queens is backtracking?
A: Huffman commits to the two least frequent nodes at each construction step; N-Queens explores choices and can undo them.

Q: Why can you split items in Fractional Knapsack but not in 0/1?
A: The problem definition permits fractional amounts only in the fractional version.

Q: Why can Quick Sort be $O(n^2)$ even though it is divide-and-conquer?
A: Divide-and-conquer alone does not guarantee balanced division; repeated unbalanced partitions produce quadratic work.

Q: Why does K-Means need numeric data?
A: Its core operation is distance to numeric centroids.

Q: Why is PCA not the clustering algorithm in your notebook?
A: The model is K-Means; PCA is applied afterward to reduce dimensions for visualization.

Q: Why not oversample the test set in churn?
A: It would change the natural evaluation distribution and make reported performance less representative.

Q: What is the biggest difference between KNN and SVM?
A: KNN predicts from nearby training examples; SVM learns a decision boundary with a margin.

Q: What if the examiner asks for a metric you did not calculate?
A: Define it correctly and say it was not explicitly calculated in this notebook.

PART XVII — FINAL NIGHT CHECKLIST

Done?	Topic
☐	I can explain every practical objective.
☐	I can explain the input → preprocessing → algorithm → output flow.
☐	I know the main time/space complexity.
☐	I can explain every important loop and decision.
☐	I can dry-run Fibonacci.
☐	I can build a small Huffman tree.
☐	I can calculate profit/weight and solve a Fractional Knapsack example.
☐	I can explain one N-Queens backtracking step.
☐	I can partition a Quick Sort example by hand.
☐	I can explain Haversine at a high level.
☐	I know MAE vs MSE vs RMSE.
☐	I know KNN's k and why scaling matters.
☐	I know SVM's C.
☐	I know what an activation function does.
☐	I know why binary cross-entropy is used for binary classification.
☐	I know class imbalance and oversampling.
☐	I know K-Means inertia and elbow.
☐	I know why PCA is used in the K-Means notebook.
☐	I know the major differences between similar algorithms.
☐	I have practiced answering aloud rather than only reading.

Golden viva rule

If you do not remember an exact implementation detail, do not invent it. Start with the principle, then connect it to the code you actually wrote. Examiners usually reward clear understanding more than memorized wording.

Ultra-short opening answers

- Fibonacci: 'Recursive uses function calls; iterative uses a loop.'
- Huffman: 'Greedy + min-heap + prefix-free binary codes.'
- Fractional Knapsack: 'Greedy by profit/weight because items are divisible.'
- N-Queens: 'Backtracking with column and diagonal safety checks.'
- Quick Sort: 'Partition around a pivot, then recursively sort both sides.'
- Uber: 'Clean → Haversine distance → outlier handling → standardize → Linear Regression → metrics.'
- Spam: 'Scale → split → KNN/SVM → confusion matrix/accuracy.'
- Churn: 'Encode → split/scale → neural network → improve imbalance/regularization → evaluate.'
- Diabetes: 'Impute invalid zeros → scale → test k → KNN → confusion matrix.'
- K-Means: 'Preprocess → elbow → k=3 → cluster → PCA visualization.'